

Key Concepts Before Starting ViT

Word	Simple Meaning	Example
Artificial Intelligence (AI)	Teaching computers to be smart like humans	A computer that recognizes your face
Deep Learning	AI that learns by looking at lots of examples, layer by layer	Learning to recognize cats by seeing 1 million cat photos
Neural Network	A system of connected layers that learns patterns	Like a team of people passing notes
Image Classification	A computer guessing what is in a photo	Seeing a photo and saying "this is a dog"
Pixel	The tiny dot that makes up a picture	A 100x100 image has 10,000 pixels
Matrix	A grid of numbers (like a spreadsheet)	A 3x3 grid of numbers
Vector	A single row or column of numbers	[1, 2, 3] is a vector

What is a Vision Transformer (ViT)?

The Vision Transformer (ViT) is a model architecture designed for computer vision tasks that applies the transformer architecture (originally developed for NLP) to images.

- **Input:** An image is split into fixed-size square patches (e.g., 16x16 pixels).
- **Patch Encoding:** Each patch is encoded into a fixed-size vector (e.g., 512 dimensions).
- **Position Encoding:** Positional information is added to each patch vector to retain spatial context.

- **Transformer Encoder:** The sequence of patch embeddings with positional encodings is passed through a transformer encoder identical to the one in "Attention Is All You Need."
- **Output:** The output includes multiple vectors; only the first vector (called the *class token*) is used.
- **Prediction Head:** This first vector is fed into a Multi-Layer Perceptron (MLP) and softmax layer to produce class probabilities for image classification.

Background & Motivation

Transformer Architecture Origins

- Introduced in 2017 for machine translation (e.g., English to French).
- Consists of an **encoder-decoder** structure.
- Encoder processes input tokens simultaneously with positional encoding.
- Decoder generates output tokens sequentially using encoded information.
- Brought state-of-the-art results in many NLP tasks (e.g., BERT, GPT).

Applying Transformers to Vision

- Inspired by NLP success, researchers aimed to apply transformers to computer vision tasks like:
 - Image classification
 - Segmentation
 - Object detection

Prior State-of-the-Art: Convolutional Neural Networks (CNNs)

- CNNs (e.g., ResNet with skip connections) were dominant for vision tasks.
- CNNs have strong **inductive biases** tailored for images.

Inductive Biases of Convolutional Networks

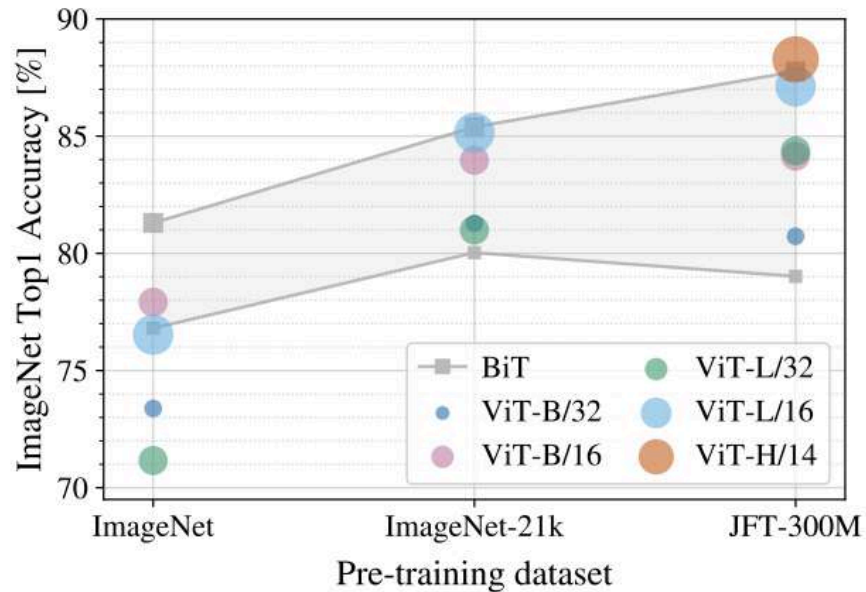
Inductive Bias	Description	Example/Mechanism
Locality	Pixels relate to their neighbors; CNNs use 2D convolutions to capture local spatial relationships.	Filters scan local patches to learn features like edges.
Translation Equivariance	Objects shifted in the image remain recognizable; CNNs apply the same filter across entire images.	Convolution operation inherently searches for patterns everywhere.
Robustness to High-Frequency Variations	Small pixel distortions shouldn't change object identity.	Pooling layers (e.g., max-pooling) summarize local info, reducing impact of noise.
Hierarchical Feature Learning	Visual processing happens in stages, from low-level features to abstract concepts.	Early CNN layers detect edges/colors; deeper layers detect complex patterns like faces or wheels.

These biases make CNNs data-efficient and powerful on moderately sized datasets (e.g., ImageNet with ~1 million images).

Performance Comparison: Vision Transformer vs ResNet

Dataset Size	Performance Summary
Small (~1 million)	ResNet outperforms Vision Transformers

Medium (~14 million)	ViT performance approaches ResNet
Large (~300 million)	Vision Transformers outperform ResNet



Key Insight:

- Scaling the pre-training dataset size benefits ViT more than CNNs.
- ViT's lack of strong inductive biases is overcome by large data availability.

Vision Transformer Architecture Details

Input Preparation

- Image size for pre-training example: pixels with 3 color channels.
- Patch size: pixels.
- Number of patches: patches.

Patch Embedding

- Each patch is flattened and passed through a shared feed-forward layer (MLP).
- Each patch is converted into a **512-dimensional vector**.
- Resulting shape: matrix.

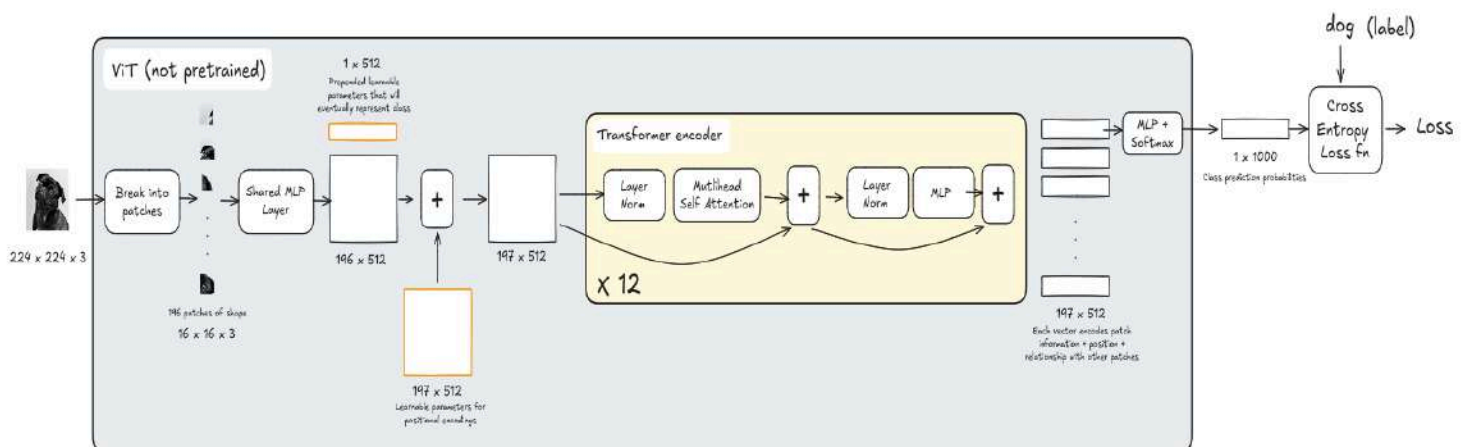
Class Token and Position Encoding

Component	Description
Class Token	A learnable vector prepended to patch embeddings that summarizes the entire image for classification.
Position Encoding	Learnable 512-dimensional vectors added to each patch embedding (including class token) to retain spatial arrangement.

The final input to the transformer encoder is a matrix.

Transformer Encoder Block

Pre-training ViT



- Stack of 12 identical layers.
- Each layer contains:
 - **Layer Normalization:** Stabilizes and accelerates training.
 - **Multi-Head Self-Attention:** Models relationships between patches.

- **Skip Connections:** Prevent degradation in deep networks.
 - **MLP with Non-linear Activation:** Enhances feature representation.
- Output shape remains , encoding patch info, positional info, and inter-patch relationships.

Classification Head

- Extract only the first vector (class token) from the output.
- Pass through an MLP to get a vector of size equal to the number of classes (e.g., 1000).
- Apply softmax to convert to class probabilities.

Training

- Use cross-entropy loss between predicted probabilities and true labels.
- Backpropagation updates:
 - Patch embedding parameters
 - Class token
 - Position encodings
 - Transformer encoder weights

Fine-Tuning Vision Transformer

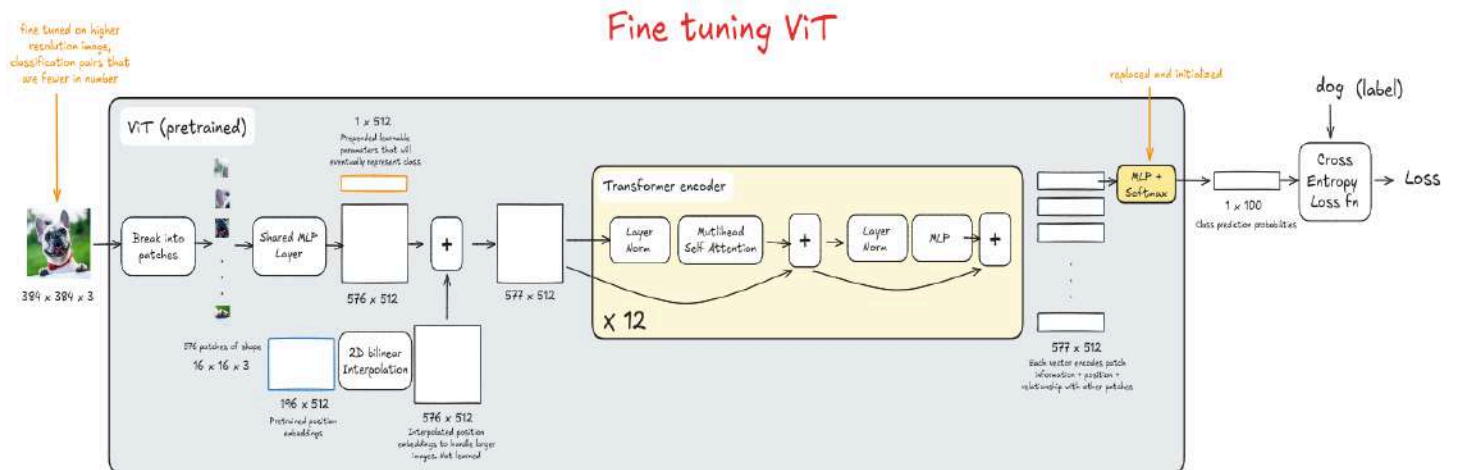
Changes for Fine-Tuning

Aspect	Pre-training	Fine-tuning
Image Resolution	224 * 224	Higher resolution, e.g.,
Number of Patches	196 patches ()	576 patches ()
Classification Head	MLP + softmax for 1000 classes	Replaced MLP + softmax for fewer classes (e.g., 100)

Handling Position Embeddings for Different Patch Counts

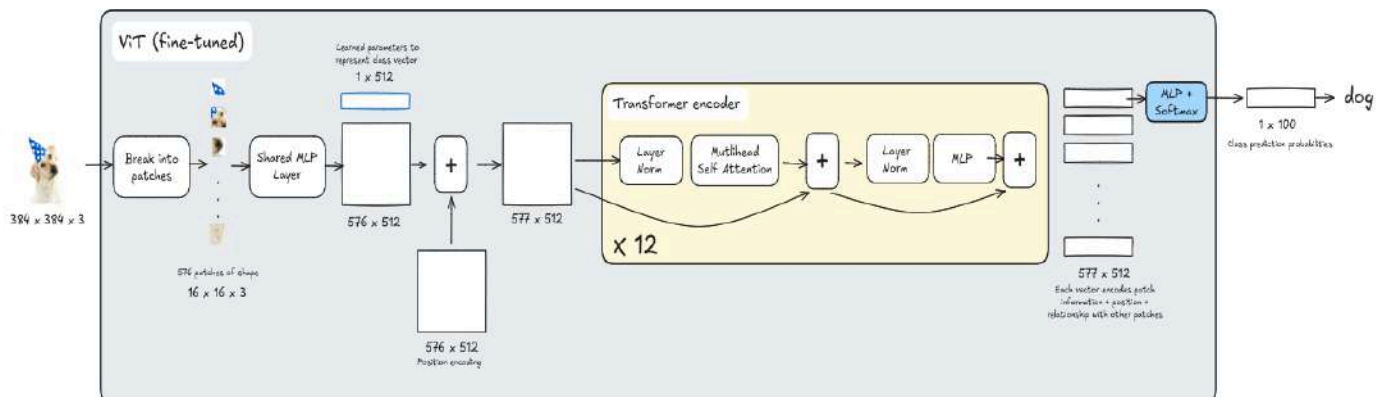
- Original position embeddings: (learned during pre-training).

- Fine-tuning requires position embeddings.
- Use **2D bilinear interpolation** to resize position embeddings:
 - Reshape to .
 - Interpolate to .
 - Flatten back to .
- This preserves spatial positional encoding while adapting to the new input size.



Inference with Vision Transformer

Inference ViT



- Input image size: e.g., .
- Break the image into patches of .
- Convert patches to d -dimensional vectors using learned patch embedding.
- Prepend the learned **class token**.
- Add interpolated **position embeddings**.
- Pass through 12-layer transformer encoder.
- Extract class token vector.
- Pass through MLP + softmax to obtain class probabilities.
- Predict class with highest probability.

Vision Transformer (ViT)

Vision Transformer (ViT) is a model that looks at an image by breaking it into small pieces called **patches**, then figures out how these pieces relate to each other to understand the image.

Metaphor: The Jigsaw Puzzle



- Imagine you have a jigsaw puzzle picture of a dog.
 - The puzzle is cut into small square pieces.
 - You look at each piece and also at how pieces connect to each other.
 - Finally, you decide: "This is a dog!"
- ViT works similarly by analyzing image patches and their relationships.

Origin

- The **Transformer** was invented in 2017 for text understanding.
- In 2020, researchers adapted this idea for images, creating ViT.
- ViT outperformed many older image recognition models.

One-Line Formula of ViT

In plain English:

- Cut image into patches
- Convert patches into numbers (embeddings)
- Model finds relationships (transformer encoder)
- Predict the class (classifier)

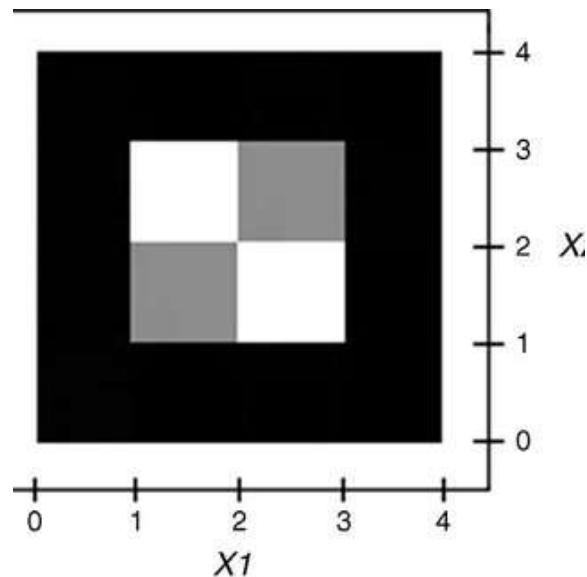
How Does ViT Work? Step-by-Step Explanation

Step	Description
1	Start with an Image: Images are grids of pixels; each pixel has a brightness/color value between 0 and 1.
2	Cut Image into Patches: Divide image into small square patches (e.g., a 4x4 image into four 2x2 patches).
3	Convert Patches to Embeddings: Multiply each flattened patch vector by an embedding matrix to get meaningful feature vectors.
4	Add [CLS] Token: A special classification token is added at the start to aggregate image info for final decision.
5	Add Positional Encoding: Adds information about patch positions so the model knows their locations.

6	Self-Attention: Every patch "looks at" every other patch to find important relationships.
7	Multi-Head Attention: Multiple attention heads run in parallel to detect different types of patterns.
8	Feed-Forward Network (MLP): Each token is processed through a small neural network for refinement.
9	Classification: The [CLS] token output is passed through a final layer to predict the image class.

Step 1: Start with an Image

Example 4x4 grayscale image pixel values:



Each number represents one pixel's brightness (0=black, 1=white).

Step 2: Cut the Image into Patches

- Image divided into four 2x2 patches.
- Each patch is flattened into a 1D vector.

Patch	Pixels	Flattened Vector
P1	0.1, 0.2, 0.5, 0.6	[0.1, 0.2, 0.5, 0.6]
P2	0.3, 0.4, 0.7, 0.8	[0.3, 0.4, 0.7, 0.8]
P3	0.9, 1.0, 1.3, 1.4	[0.9, 1.0, 1.3, 1.4]
P4	1.1, 1.2, 1.5, 1.6	[1.1, 1.2, 1.5, 1.6]

Step 3: Convert Patches to Embeddings

Why embeddings? Raw pixel values are not informative enough; embeddings capture important features like edges and textures.

- Each patch vector () is multiplied by an embedding matrix ()

Example for Patch 1:

The full embedding matrix for all patches becomes:

As shown in the figure below:

Step 4: Add [CLS] Token

- A special token called **[CLS]** is added at the start of the sequence.
- This token aggregates information and is used for classification at the end.

Sequence after adding [CLS]:

Token	Vector
-------	--------

[CLS]	[0, 0, 1]
Patch 1	[1.04, 1.18, 1.32]
Patch 2	[1.48, 1.70, 1.92]
Patch 3	[2.80, 3.26, 3.72]
Patch 4	[3.24, 3.78, 4.32]

Step 5: Add Positional Encoding

Why? The transformer does not inherently know the position of patches in the image.

- Positional encodings are added to each token vector to encode their spatial location.
- **Analogy:** Like putting numbered stickers on puzzle pieces to remember their place.

Step 6: Self-Attention — The Core of ViT

Self-attention allows each patch to "look at" every other patch to assess their importance in understanding the image.

Restaurant Metaphor for Attention:

Symbol	Meaning	Explanation
Q	Query	What am I looking for?

K	Key	What does each item offer?
V	Value	The actual item or information

- You ask: "I want something spicy and vegetarian" (Query)
- Menu says: "Pad Thai: spicy, Thai, vegetarian" (Key)
- You get: The Pad Thai dish (Value)

In ViT:

- Each patch queries which other patches are important (query).
- Each patch offers information (key).
- Actual information passed is the value.

Attention Formula:

Symbol	Meaning	Simple Explanation
Q	Query	What am I looking for?
K	Key	What do I have to offer?
V	Value	The actual information
K^T	Transpose of	Flip the Key matrix (rows $\rightarrow$ columns)

sqrt(key dimension)	Square root of key dimension	Stabilizes the scale of dot-products
softmax	Converts to probabilities	Attention weights sum to 1.0

Step 7: Multi-Head Attention (MSA)

- Instead of one attention, ViT performs **multiple attention heads in parallel**.
- Each head learns different patterns (edges, textures, colors).
- Heads are concatenated and projected to produce the final output.

Formula:MSA(Z) = Concat(head1, head2, ...) x W_o

Where _o is a learned weight matrix.

Step 8: Feed-Forward Network (MLP)

- Each token (including [CLS]) passes through a small neural network to refine features.
- The MLP uses GELU activation, which smoothly controls information flow:
- GELU acts like a gate that opens more for important signals and less for unimportant ones.

Step 9: Classification — Final Decision

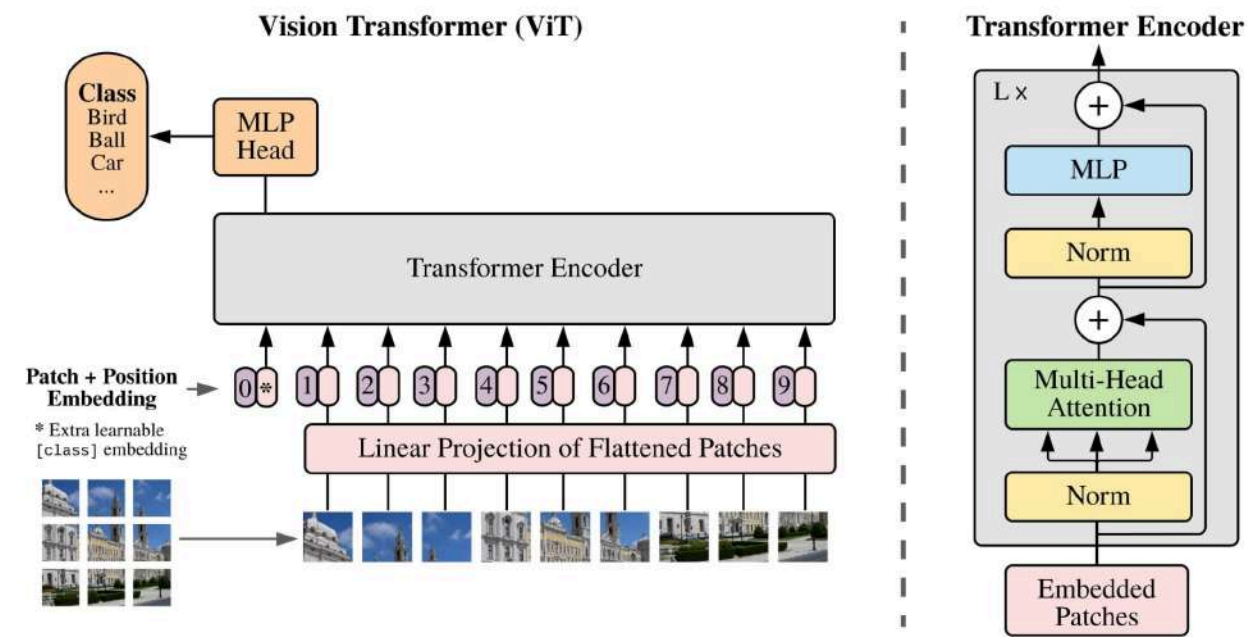
- After multiple transformer layers, the **[CLS] token** holds the summary representation of the entire image.
- It passes through a final classification layer:
- The output is a probability distribution over classes.
- The class with the highest probability is predicted.

Example:

CLS Token Vector	[-0.105, 0.050, 2.330]
Raw Scores (logits)	Class 1 = 1.25, Class 2 = 0.52

Softmax Probabilities	Class 1 = 67.6%, Class 2 = 32.4%
Prediction	Class 1 (highest probability)

Full ViT Architecture — Big Picture



Simple Step-by-Step Math Example

Step	Description	Example and Values
1	Image pixels (4x4 grayscale)	See pixel matrix in Step 1
2	Extract 4 patches (2x2 each)	Flattened patches in Step 2
3	Linear embedding via matrix multiplication	Embedding matrix in Step 3

4	Add [CLS] token and positional encoding	Sequence of 5 tokens with added position info
5	Layer normalization (mean=0, std=1)	Normalizing each token vector (see below)
6	Self-attention calculation	Project vectors to Q, K, V and compute scores

Real-Life Applications of Vision Transformer

Application	What ViT Does	Example
Image Classification	Identifies objects in images	Photo of a cat → 'Cat, 97% confidence'
Medical Imaging	Detects diseases in MRI/CT scans	Finds tumors missed by doctors
Autonomous Vehicles	Understands road scenes from camera feeds	Detects stop signs, pedestrians, cars
Satellite Imagery	Analyzes land types from space photos	Differentiates forest, farmland, city
CLIP (Multimodal)	Matches images with text descriptions	Given "a dog running", finds matching image

ViT vs CNN — Key Differences

Feature	CNN (Convolutional Neural Network)	ViT (Vision Transformer)
How it processes images	Scans small regions sequentially	Looks at all patches simultaneously
Global understanding	Gradual through layers	Available from the first layer
Data requirements	Works well with small datasets	Needs large datasets to perform best
Speed on GPU	Very fast	Fast but requires more memory
Flexibility	Best for images only	Handles images, text, and audio
Interpretability	Hard to interpret	Attention maps reveal what model focuses on

Advantages and Disadvantages of ViT

Advantages	Disadvantages
Understands global context from the first layer	Requires huge training datasets

Scales well: bigger model + more data = better	High computational cost (attention is)
Same design works for images, text, and audio	Fixed patch size might miss very small objects
Attention maps improve interpretability	Training can be unstable without careful tuning
Outperforms CNN on large-scale benchmarks	Slower inference compared to small CNNs.

Key Symbols & Glossary

Symbol / Term	Full Name	Simple Meaning
H, W, C	Height, Width, Channels	Dimensions of the input image (e.g., 224x224x3 for RGB)
P	Patch size	Side length of each square patch (e.g., 16)
N	Number of patches	
D	Embedding dimension	Length of each patch vector after projection (e.g., 768)
E	Embedding matrix	Learnable matrix projecting patches into -dimensional space
E_{pos}	Positional embedding	Learnable matrix encoding position of each token

z_{cls}	CLS token	Summary token used for final classification
Q, K, V	Query, Key, Value	Three projections of input used in attention
d_k	Key dimension	Dimensionality of Q and K; used in scaling attention scores
MSA	Multi-Head Self-Attention	Attention computed in parallel across multiple heads
LN	Layer Normalization	Normalizes token features to zero mean and unit variance
MLP	Multi-Layer Perceptron	Two-layer feed-forward network inside each encoder block
GELU	Gaussian Error Linear Unit	Smooth activation function used in MLP
y-hat	Predicted class	Output class label with highest softmax probability
W_O	Output projection	Matrix combining multi-head outputs
softmax	Softmax function	Converts scores to probabilities summing to 1.0

Quiz Question: Inductive Biases in Vision Transformers

Question: What are some inductive biases that vision transformers make about images?

Option	Description	Correct?
A	Converting an image to a 2D grid of patches.	Yes
B	Using convolution and pooling like operations.	No
C	Using skip connections similar to ResNet.	No
D	The 2D interpolation of position embeddings.	Yes

Key Takeaways

- Vision Transformer adapts the original transformer architecture from NLP to vision by treating an image as a sequence of patches.
- CNNs outperform ViTs on smaller datasets due to strong inductive biases (locality, translation equivariance, robustness, hierarchical features).
- ViTs outperform CNNs when trained on very large datasets due to their scalable architecture without hard-coded biases.
- Position embeddings are critical and adapted via interpolation when changing input resolution during fine-tuning.
- Pre-training, fine-tuning, and inference pipelines all involve patch embedding, positional encoding, transformer encoding, and classification heads.

Sources:

 Vision Transformers - Explained!

<https://link.excalidraw.com/p/readonly/6S2vdCqfNqdfLTeyJ3UB>